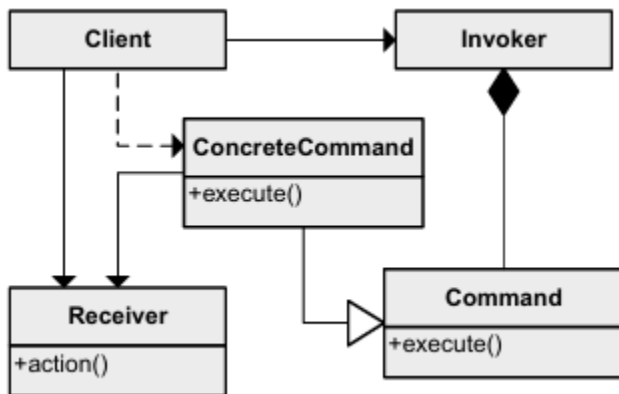
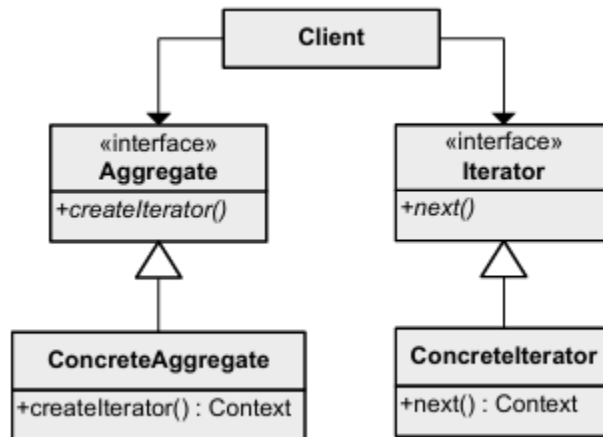


Annexe

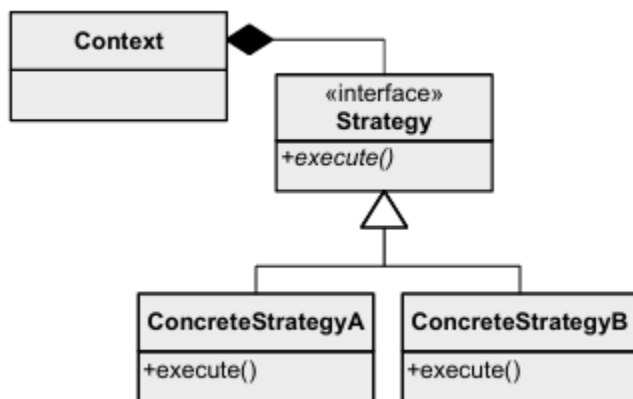
Command



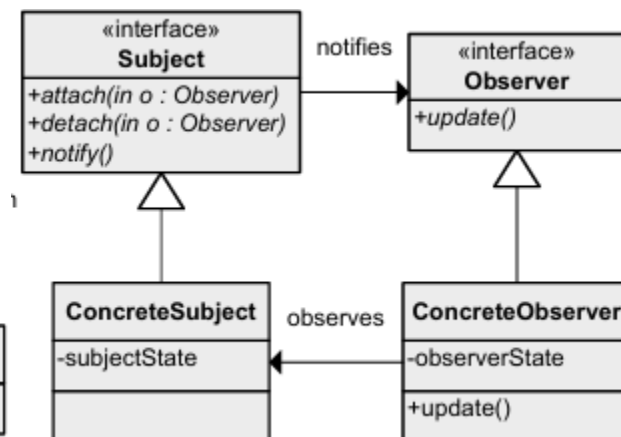
Iterator



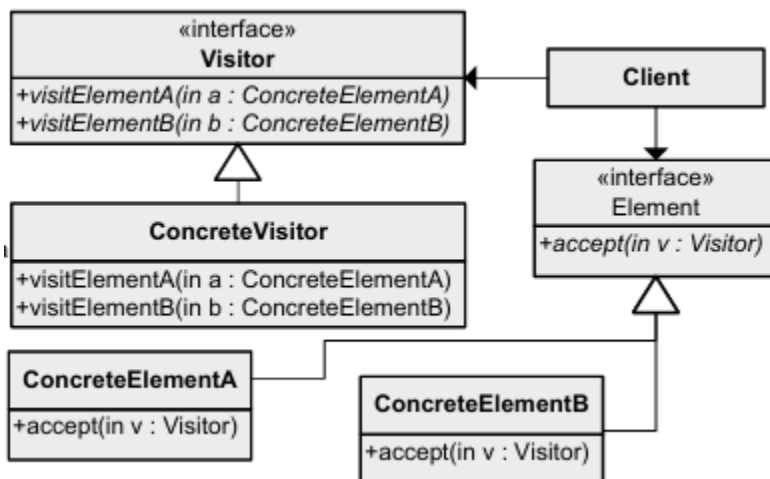
Strategy



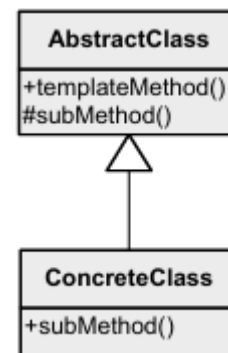
Observer



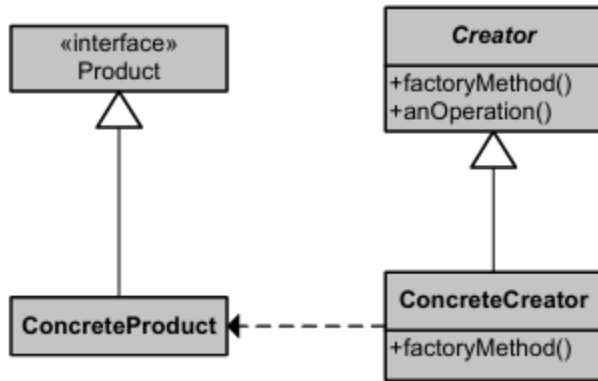
Visitor



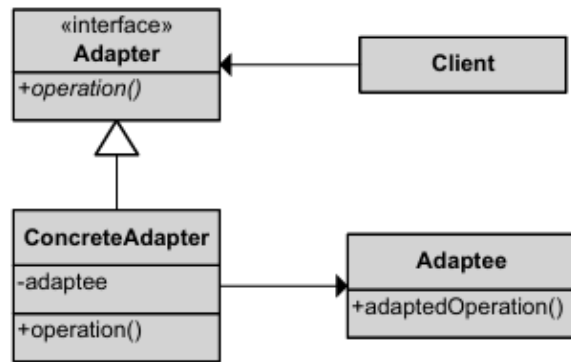
Template Method



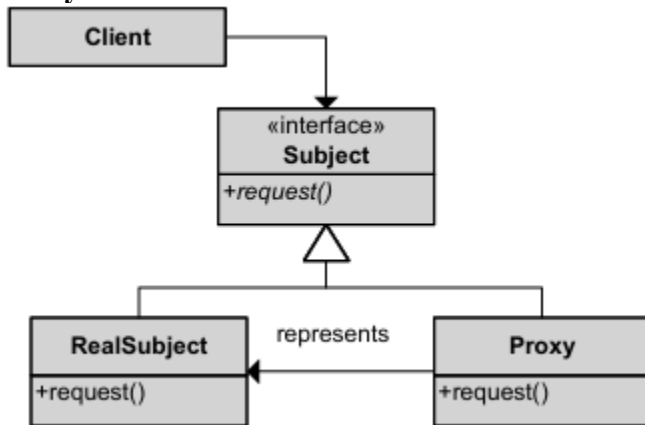
Factory Method



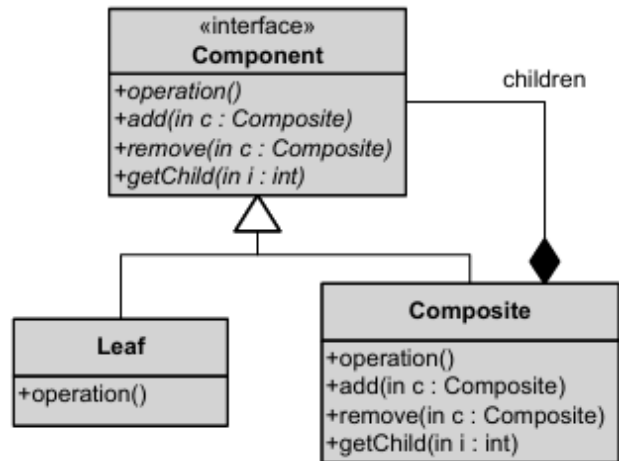
Adapter



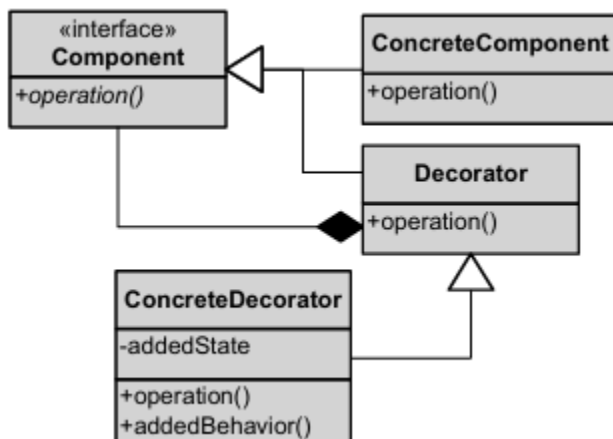
Proxy



Composite



Decorator



Guide détaillé des Design Patterns (NFP121)

Ce document explique en détail chacun des 11 patterns de conception présentés en annexe, avec leur intention, leur structure, et un exemple de code Java pour chacun.

1. Command (Commande)

Intention

Encapsuler une requête (une action) sous forme d'objet, afin de pouvoir la paramétrer, la mettre en file d'attente, l'annuler (undo), ou la journaliser. On découple ainsi celui qui **demande** une action (l'Invoker) de celui qui **sait l'exécuter** (le Receiver).

Quand l'utiliser

- Pour implémenter des actions « annulables » (undo/redo), comme dans un éditeur de texte.
- Pour faire des files d'attente de tâches (queues de commandes, jobs à exécuter plus tard).
- Pour découpler un menu/bouton de l'action concrète qu'il déclenche (ex : un bouton « Imprimer » ne connaît pas l'imprimante, il appelle juste `command.execute()`).

Rôles (UML)

- **Command** : interface avec une méthode `execute()`.
- **ConcreteCommand** : implémente `execute()` et délègue au Receiver.
- **Receiver** : sait réellement effectuer le travail (`action()`).
- **Invoker** : déclenche la commande sans connaître son implémentation.
- **Client** : crée le ConcreteCommand et l'associe à son Receiver.

Exemple Java

```
// Receiver : sait faire le travail réel
public class Lumiere {
    public void allumer() { System.out.println("Lumière allumée"); }
    public void eteindre() { System.out.println("Lumière éteinte"); }
}

// Command
public interface Command {
    void execute();
}

// ConcreteCommand
public class AllumerLumiereCommand implements Command {
    private Lumiere lumiere;
    public AllumerLumiereCommand(Lumiere lumiere) { this.lumiere = lumiere; }
    @Override
    public void execute() { lumiere.allumer(); }
}

// Invoker
public class Telecommande {
    private Command command;
    public void setCommand(Command command) { this.command = command; }
    public void appuyerSurBouton() { command.execute(); }
}

// Client
public class Main {
    public static void main(String[] args) {
        Lumiere salon = new Lumiere();
        Command allumer = new AllumerLumiereCommand(salon);

        Telecommande telecommande = new Telecommande();
        telecommande.setCommand(allumer);
        telecommande.appuyerSurBouton(); // -> "Lumière allumée"
    }
}
```

2. Iterator (Itérateur)

Intention

Fournir un moyen d'accéder séquentiellement aux éléments d'une collection **sans exposer sa représentation interne** (tableau, liste chaînée, arbre...).

Quand l'utiliser

- Quand on veut parcourir une collection personnalisée (par ex. une structure de données « maison ») de façon uniforme, comme avec un `for-each` Java.
- Quand on veut pouvoir changer l'implémentation interne de la collection sans casser le code qui la parcourt.

Rôles (UML)

- **Aggregate** : interface définissant `createIterator()`.
- **ConcreteAggregate** : la collection concrète.
- **Iterator** : interface avec `next()`, souvent aussi `hasNext()`.
- **ConcreteIterator** : connaît la position courante dans la collection.

Exemple Java

```
import java.util.*;

public class Equipe implements Iterable<String> {
    private List<String> joueurs = new ArrayList<>();

    public void ajouter(String joueur) { joueurs.add(joueur); }

    @Override
    public Iterator<String> iterator() {
        return new EquipeIterator();
    }

    // ConcreteIterator interne
    private class EquipeIterator implements Iterator<String> {
        private int index = 0;
        @Override
        public boolean hasNext() { return index < joueurs.size(); }
        @Override
        public String next() { return joueurs.get(index++); }
    }
}

public class Main {
    public static void main(String[] args) {
        Equipe equipe = new Equipe();
        equipe.ajouter("Ali");
        equipe.ajouter("Sara");

        for (String joueur : equipe) { // utilise notre Iterator
            System.out.println(joueur);
        }
    }
}
```

Remarque : en Java, on utilise rarement à la main `Iterator`, car le langage le fournit déjà via `Iterable/Collection`. Mais comprendre le pattern reste essentiel (c'est ce qui se cache derrière `for-each`).

3. Strategy (Stratégie)

Intention

Définir une **famille d'algorithmes interchangeable**, encapsuler chacun d'eux, et les rendre interchangeables à l'exécution sans changer le code du Client.

Quand l'utiliser

- Quand plusieurs algorithmes font la même chose mais différemment (ex : plusieurs façons de trier, de calculer une remise, de valider un mot de passe).
- Pour remplacer une cascade de `if/else` ou `switch` qui choisit un comportement.

Rôles (UML)

- **Strategy** : interface commune (`execute()`).
- **ConcreteStrategyA/B** : implémentations différentes de l'algorithme.
- **Context** : utilise une Strategy par composition, sans connaître son implémentation.

Exemple Java

```
// Strategy
public interface StrategiePaiement {
    void payer(double montant);
}

// ConcreteStrategyA
public class PaiementCarte implements StrategiePaiement {
    @Override
    public void payer(double montant) {
        System.out.println("Paiement de " + montant + " par carte bancaire");
    }
}

// ConcreteStrategyB
public class PaiementPaypal implements StrategiePaiement {
    @Override
    public void payer(double montant) {
        System.out.println("Paiement de " + montant + " via PayPal");
    }
}

// Context
public class Commande {
    private StrategiePaiement strategie;
    public void setStrategiePaiement(StrategiePaiement strategie) {
        this.strategie = strategie;
    }
    public void payer(double montant) {
        strategie.payer(montant);
    }
}

public class Main {
    public static void main(String[] args) {
        Commande commande = new Commande();
        commande.setStrategiePaiement(new PaiementCarte());
        commande.payer(100.0); // -> paiement par carte

        commande.setStrategiePaiement(new PaiementPaypal());
        commande.payer(50.0); // -> paiement par PayPal
    }
}
```

4. Observer (Observateur)

Intention

Définir une dépendance **un-à-plusieurs** entre objets, de sorte que lorsqu'un objet (le Subject) change d'état, tous ses dépendants (les Observers) soient notifiés et mis à jour automatiquement.

Quand l'utiliser

- Système d'abonnement/notification (newsletters, notifications push, événements UI).
- Quand plusieurs objets doivent réagir à un changement d'état sans être fortement couplés à l'objet qui change.

Rôles (UML)

- **Subject** : interface avec `attach()`, `detach()`, `notify()`.
- **ConcreteSubject** : maintient l'état et la liste des observateurs.
- **Observer** : interface avec `update()`.
- **ConcreteObserver** : réagit au changement (`update()`).

Exemple Java

```
import java.util.*;
```

```

// Observer
public interface Observer {
    void update(String message);
}

// Subject
public interface Subject {
    void attach(Observer o);
    void detach(Observer o);
    void notifyObservers(String message);
}

// ConcreteSubject
public class ChaîneYoutube implements Subject {
    private List<Observer> abonnes = new ArrayList<>();

    @Override
    public void attach(Observer o) { abonnes.add(o); }
    @Override
    public void detach(Observer o) { abonnes.remove(o); }
    @Override
    public void notifyObservers(String message) {
        for (Observer o : abonnes) o.update(message);
    }

    public void publierVideo(String titre) {
        notifyObservers("Nouvelle vidéo : " + titre);
    }
}

// ConcreteObserver
public class Abonne implements Observer {
    private String nom;
    public Abonne(String nom) { this.nom = nom; }
    @Override
    public void update(String message) {
        System.out.println(nom + " a reçu : " + message);
    }
}

public class Main {
    public static void main(String[] args) {
        ChaîneYoutube chaine = new ChaîneYoutube();
        chaine.attach(new Abonne("Ali"));
        chaine.attach(new Abonne("Sara"));

        chaine.publierVideo("Design Patterns expliqués");
        // -> Ali a reçu : Nouvelle vidéo : ...
        // -> Sara a reçu : Nouvelle vidéo : ...
    }
}

```

C'est exactement le pattern à utiliser pour l'Exercice 3 de votre examen (abonnement/désabonnement aux notifications).

5. Visitor (Visiteur)

Intention

Représenter une opération à effectuer sur les éléments d'une structure d'objets **sans modifier les classes de ces éléments**. Permet d'ajouter facilement de nouvelles opérations sans toucher aux classes existantes.

Quand l'utiliser

- Quand on a une hiérarchie d'objets stable mais qu'on veut souvent ajouter de **nouvelles opérations** sur cette hiérarchie (ex : calculer le prix, exporter en XML, afficher...).
- Typique pour les AST (arbres syntaxiques), les structures de documents, etc.

Rôles (UML)

- **Element** : interface avec `accept(Visitor v)`.
- **ConcreteElementA/B** : implémentent `accept()` qui appelle `visitor.visitElementA(this)`.
- **Visitor** : interface avec une méthode `visit` par type d'élément concret.
- **ConcreteVisitor** : implémente le comportement réel pour chaque type.

Exemple Java

```

// Element
public interface Forme {
    void accept(VisiteurForme visiteur);
}

public class Cercle implements Forme {
    public double rayon = 5;
    @Override
    public void accept(VisiteurForme visiteur) { visiteur.visiterCercle(this); }
}

public class Carre implements Forme {
    public double cote = 4;
    @Override
    public void accept(VisiteurForme visiteur) { visiteur.visiterCarre(this); }
}

// Visitor
public interface VisiteurForme {
    void visiterCercle(Cercle c);
    void visiterCarre(Carre c);
}

// ConcreteVisitor : calcule l'aire
public class CalculAire implements VisiteurForme {
    @Override
    public void visiterCercle(Cercle c) {
        System.out.println("Aire cercle = " + (Math.PI * c.rayon * c.rayon));
    }
    @Override
    public void visiterCarre(Carre c) {
        System.out.println("Aire carré = " + (c.cote * c.cote));
    }
}

public class Main {
    public static void main(String[] args) {
        List<Forme> formes = List.of(new Cercle(), new Carre());
        VisiteurForme calcul = new CalculAire();
        for (Forme f : formes) f.accept(calcul);
    }
}

```

Avantage : pour ajouter une nouvelle opération (ex : AfficherForme), on crée un nouveau Visitor — aucune classe Forme n'est modifiée.

6. Template Method (Patron de méthode)

Intention

Définir le **squelette d'un algorithme** dans une méthode, en laissant certaines étapes à redéfinir par les sous-classes, sans changer la structure générale de l'algorithme.

Quand l'utiliser

- Quand plusieurs classes partagent le même déroulement global, mais que certaines étapes varient.
- Évite la duplication de code entre sous-classes similaires.

Rôles (UML)

- **AbstractClass** : définit `templateMethod()` (final, non modifiable) qui appelle des sous-méthodes, dont certaines abstraites (`subMethod()`).
- **ConcreteClass** : redéfinit seulement les étapes variables.

Exemple Java

```

public abstract class PreparationBoisson {
    // Template method : le squelette est fixe
    public final void preparer() {
        bouillirEau();
        ajouterIngredientPrincipal();
        verserDansTasse();
        ajouterCondiment();
    }

    private void bouillirEau() { System.out.println("Eau bouillie"); }
    private void verserDansTasse() { System.out.println("Versé dans la tasse"); }

    // Étapes variables, à redéfinir
}

```

```

        protected abstract void ajouterIngredientPrincipal();
        protected abstract void ajouterCondiment();
    }

    public class PreparationThe extends PreparationBoisson {
        @Override
        protected void ajouterIngredientPrincipal() { System.out.println("Ajout du thé"); }
        @Override
        protected void ajouterCondiment() { System.out.println("Ajout de citron"); }
    }

    public class PreparationCafe extends PreparationBoisson {
        @Override
        protected void ajouterIngredientPrincipal() { System.out.println("Ajout du café"); }
        @Override
        protected void ajouterCondiment() { System.out.println("Ajout de sucre"); }
    }

    public class Main {
        public static void main(String[] args) {
            new PreparationThe().preparer();
            new PreparationCafe().preparer();
        }
    }

```

7. Factory Method (Fabrique)

Intention

Définir une **interface pour créer un objet**, mais laisser les sous-classes décider quelle classe concrète instancier. Découple le code client de la création des objets concrets.

Quand l'utiliser

- Quand une classe ne peut pas savoir à l'avance quel type concret d'objet elle doit créer.
- Pour respecter le principe Ouvert/Fermé : ajouter un nouveau type de produit sans modifier le code existant qui utilise la fabrique.

Rôles (UML)

- **Product** : interface du produit créé.
- **ConcreteProduct** : implémentation concrète.
- **Creator** : déclare `factoryMethod()` (souvent abstraite).
- **ConcreteCreator** : implémente `factoryMethod()` et retourne un `ConcreteProduct`.

Exemple Java

```

// Product
public interface Vehicule {
    void rouler();
}

public class Voiture implements Vehicule {
    @Override
    public void rouler() { System.out.println("La voiture roule"); }
}

public class Moto implements Vehicule {
    @Override
    public void rouler() { System.out.println("La moto roule"); }
}

// Creator
public abstract class VehiculeFactory {
    public abstract Vehicule creerVehicule();

    public void livrer() {
        Vehicule v = creerVehicule();
        v.rouler();
    }
}

// ConcreteCreator
public class VoitureFactory extends VehiculeFactory {
    @Override
    public Vehicule creerVehicule() { return new Voiture(); }
}

public class MotoFactory extends VehiculeFactory {

```



```

@Override
public Vehicule creerVehicule() { return new Moto(); }
}

public class Main {
    public static void main(String[] args) {
        VehiculeFactory factory = new VoitureFactory();
        factory.livrer(); // -> "La voiture roule"
    }
}

```

8. Adapter (Adaptateur)

Intention

Convertir l'interface d'une classe existante (souvent une classe externe/non modifiable) en une autre interface attendue par le client, **sans modifier le code de la classe adaptée**.

Quand l'utiliser

- Pour intégrer une bibliothèque externe dont l'interface ne correspond pas à celle de votre application.
- Pour faire collaborer du code legacy avec du code nouveau.

Rôles (UML)

- **Adapter** (interface cible attendue par le Client).
- **ConcreteAdapter** : implémente Adapter, contient une référence vers l'Adaptee, et traduit les appels.
- **Adaptee** : la classe existante incompatible (qu'on ne modifie pas).

Exemple Java

(directement lié à l'Exercice 1 de votre examen)

```

// Adaptee externe non modifiable
public class ExternalJsonParser {
    public String parseJson(String file) {
        return "Contenu JSON du fichier " + file;
    }
}

// Target : interface attendue
public interface FileReader {
    String read(String filePath);
}

// ConcreteAdapter
public class JsonReaderAdapter implements FileReader {
    private ExternalJsonParser parser;
    public JsonReaderAdapter(ExternalJsonParser parser) { this.parser = parser; }

    @Override
    public String read(String filePath) {
        return parser.parseJson(filePath); // traduit l'appel
    }
}

public class Main {
    public static void main(String[] args) {
        FileReader reader = new JsonReaderAdapter(new ExternalJsonParser());
        System.out.println(reader.read("data.json"));
    }
}

```

9. Proxy

Intention

Fournir un **objet de substitution (intermédiaire)** qui contrôle l'accès à un autre objet (le RealSubject), en ajoutant un comportement avant/après l'appel : contrôle d'accès, mise en cache, journalisation, chargement différé (lazy loading), etc.

Quand l'utiliser

- **Proxy de cache** : éviter de refaire un calcul/chargement coûteux déjà effectué (exactement le besoin de l'Exercice 1 : éviter de relire un fichier déjà lu).
- **Proxy de protection** : vérifier les droits d'accès avant d'appeler le RealSubject.
- **Proxy virtuel** : créer l'objet réel seulement quand c'est nécessaire (lazy loading).

Rôles (UML)

- **Subject** : interface commune entre RealSubject et Proxy.
- **RealSubject** : l'objet réel qui fait le travail.
- **Proxy** : implémente la même interface, garde une référence vers RealSubject, et ajoute un comportement supplémentaire.

Exemple Java

(directement lié à l'Exercice 1 : Proxy de cache pour FileReader)

```
import java.util.*;

public interface FileReader {
    String read(String filePath);
}

// RealSubject (ou adapté via Adapter, comme JsonReaderAdapter)
public class CsvReader implements FileReader {
    @Override
    public String read(String filePath) {
        return "Contenu CSV du fichier " + filePath;
    }
}

// Proxy de cache
public class CachedFileReaderProxy implements FileReader {
    private FileReader reader; // RealSubject (ou Adapter)
    private Map<String, String> cache = new HashMap<>();

    public CachedFileReaderProxy(FileReader reader) { this.reader = reader; }

    @Override
    public String read(String filePath) {
        if (cache.containsKey(filePath)) {
            System.out.println("Lecture depuis le cache pour " + filePath);
            return cache.get(filePath);
        }
        String contenu = reader.read(filePath); // appel réel, coûteux
        cache.put(filePath, contenu);
        return contenu;
    }
}

public class Main {
    public static void main(String[] args) {
        FileReader reader = new CachedFileReaderProxy(new CsvReader());
        System.out.println(reader.read("data.csv")); // lecture réelle
        System.out.println(reader.read("data.csv")); // lecture depuis le cache
    }
}
```

10. Composite (Composite)

Intention

Composer des objets en **structures arborescentes** pour représenter des hiérarchies tout-partie (part-whole), et permettre au client de traiter de manière **uniforme** un objet simple (Leaf) et un groupe d'objets (Composite).

Quand l'utiliser

- Structures arborescentes : fichiers/dossiers, menus imbriqués, organigrammes.
- **Groupes de destinataires imbriqués** (département → équipe → sous-équipe), exactement le cas de l'Exercice 3 de votre examen.

Rôles (UML)

- **Component** : interface commune avec `operation()` (et souvent `add()`, `remove()`, `getChild()`).
- **Leaf** : élément simple, sans enfants.
- **Composite** : contient une liste de Component (enfants), implémente `operation()` en la propageant à tous ses enfants.

Exemple Java

```
import java.util.*;

// Component
public interface Destinataire {
    void notifier(String message);
}

// Leaf
public class Utilisateur implements Destinataire {
    private String nom;
    public Utilisateur(String nom) { this.nom = nom; }
    @Override
    public void notifier(String message) {
        System.out.println(nom + " a reçu : " + message);
    }
}

// Composite
public class Groupe implements Destinataire {
    private String nomGroupe;
    private List<Destinataire> membres = new ArrayList<>();

    public Groupe(String nomGroupe) { this.nomGroupe = nomGroupe; }
    public void ajouter(Destinataire d) { membres.add(d); }
    public void retirer(Destinataire d) { membres.remove(d); }

    @Override
    public void notifier(String message) {
        for (Destinataire d : membres) {
            d.notifier(message); // propage récursivement (Leaf ou Composite)
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Utilisateur ali = new Utilisateur("Ali");
        Utilisateur sara = new Utilisateur("Sara");

        Groupe sousEquipe = new Groupe("Sous-équipe Backend");
        sousEquipe.ajouter(ali);
        sousEquipe.ajouter(sara);

        Groupe equipe = new Groupe("Équipe Dev");
        equipe.ajouter(sousEquipe); // un groupe peut contenir un autre groupe !
        equipe.ajouter(new Utilisateur("Karim"));

        equipe.notifier("Réunion à 15h"); // notifie tout le monde, récursivement
    }
}
```

11. Decorator (Décorateur)

Intention

Ajouter dynamiquement des **responsabilités supplémentaires** à un objet, sans modifier sa classe ni utiliser l'héritage massif. Plusieurs décorateurs peuvent s'empiler.

Quand l'utiliser

- Quand on veut **combinaison plusieurs options/comportements** sur un objet de base, de façon flexible (ex : café + lait + sucre).
- Exactement le besoin de l'Exercice 2 : un service (CLOUD, STORAGE...) peut être enrichi par des options combinables (`urgent`, `highSecurity`) sans créer une sous-classe pour chaque combinaison.

Rôles (UML)

- **Component** : interface commune (`operation()`).
- **ConcreteComponent** : objet de base.

- **Decorator** : implémente Component, contient une référence vers un Component (composition), délègue puis ajoute du comportement.
- **ConcreteDecorator** : ajoute un comportement précis (`addedBehavior()`).

Exemple Java

```
// Component
public interface Service {
    double getPrix();
    String getDescription();
}

// ConcreteComponent
public class ServiceCloud implements Service {
    private int quantite;
    public ServiceCloud(int quantite) { this.quantite = quantite; }
    @Override
    public double getPrix() { return quantite * 10; }
    @Override
    public String getDescription() { return "Service CLOUD"; }
}

// Decorator abstrait
public abstract class ServiceDecorator implements Service {
    protected Service service; // composition vers le Component décoré
    public ServiceDecorator(Service service) { this.service = service; }
}

// ConcreteDecorator : option urgente
public class OptionUrgente extends ServiceDecorator {
    public OptionUrgente(Service service) { super(service); }
    @Override
    public double getPrix() { return service.getPrix() * 1.05; } // +5%
    @Override
    public String getDescription() { return service.getDescription() + " + urgent"; }
}

// ConcreteDecorator : option haute sécurité
public class OptionHighSecurity extends ServiceDecorator {
    public OptionHighSecurity(Service service) { super(service); }
    @Override
    public double getPrix() { return service.getPrix() * 1.06; } // +6%
    @Override
    public String getDescription() { return service.getDescription() + " + highSecurity"; }
}

public class Main {
    public static void main(String[] args) {
        Service service = new ServiceCloud(10);
        service = new OptionUrgente(service); // on empile les décorateurs
        service = new OptionHighSecurity(service); // combinables librement

        System.out.println(service.getDescription());
        System.out.println(service.getPrix());
    }
}
```

Tableau récapitulatif

Pattern	Catégorie	Problème résolu	Mot-clé
Command	Comportemental	Encapsuler une action en objet	undo, queue de tâches
Iterator	Comportemental	Parcourir une collection sans exposer sa structure	for-each
Strategy	Comportemental	Choisir un algorithme à l'exécution	interchangeable
Observer	Comportemental	Notifier automatiquement des dépendants	abonnement/notification
Visitor	Comportemental	Ajouter des opérations sans modifier les classes	nouvelle opération
Template Method	Comportemental	Squelette fixe, étapes variables	héritage, étapes
Factory Method	Création	Déléguer la création d'objets aux sous-classes	création flexible

Adapter	Structurel	Rendre compatibles deux interfaces incompatibles	bibliothèque externe
Proxy	Structurel	Contrôler l'accès / ajouter un comportement transparent	cache, sécurité
Composite	Structurel	Traiter uniformément un objet seul ou un groupe	arborescence
Decorator	Structurel	Ajouter des responsabilités combinables dynamiquement	options empilables

Lien direct avec votre examen

- **Exercice 1** : Adapter (pour ExternalJsonParser/ExternalXmlParser) + Proxy (cache pour éviter de relire un fichier déjà lu).
- **Exercice 2** : Strategy ou Factory Method pour les types de services, Decorator pour les options combinables (urgent, highSecurity), Strategy pour les formats de sortie (PDF, HTML).
- **Exercice 3** : Composite pour les destinataires (utilisateur/groupe imbriqué) + Observer pour l'abonnement/désabonnement aux notifications.